Slack Task Manager — Product Spec

Version 1 · Product-facing, non-technical For a developer-facing technical reference see `SPEC.md`.

---

1. What this agent is

A Slack-native AI task manager. The bot lives inside Slack, listens to messages, and turns conversations into tracked tasks without anyone leaving the chat. Tasks land in a shared Google Sheet in real time, and people get plans and reminders pushed into their Slack DMs.

2. Why it exists

Most teams already coordinate work in Slack threads, voice notes, and DMs. Tasks discussed there get forgotten because moving them into Jira is friction nobody pays. This bot removes the friction:

- It **captures the task in place** (mention, shortcut, voice, or passive detection of "we need to do X by Friday").
- It **tracks the lifecycle in place** (buttons on the Slack card — Start, Mark done, Cancel, Edit, Delete).
- It **reports the state in place** (DMs for digests + reminders, plus a live-synced Google Sheet for managers).

The team works where it already is; managers get visibility through the sheet and DM digests they don't have to chase.

3. User roles

The bot serves four roles. Most people fill several at once.

Role	What's in it for them
Contributor (anyone in a Slack workspace)	Captures tasks from chat in two clicks; never has to type things into another tool. Gets a daily plan in DM, gets reminders before deadlines slip.
Task owner (the assignee of a specific task)	Sees clean cards with Start/Mark done/Cancel/Edit. Decides when to start, when to drop, what to attach as a result. Doesn't get pinged by digests for tasks that aren't theirs.
Subscriber (anyone watching someone else's task)	Sees the task in their *Tracking* section, gets DM updates when the status changes — without owning the work. Useful for stakeholders, dependent reviewers, leadership.
Admin / manager	Sees the whole team's open work in one DM each morning + evening. Can edit, reassign, or delete any task. Sees a real-time Google Sheet with every task's current state and history.

There's also one off-stage role:

Role	What's in it for them
Operator (the person who runs the bot)	Configures Slack tokens, OpenAI/Anthropic key, Google Service Account; the bot then runs unattended in Docker on a single VM.

---

4. Features

Each feature lists the user stories it serves and a step-by-step flow for each story. "Flows" describe the user-visible sequence; the bot's internal mechanics are deliberately omitted.

Feature 1 — Task capture

The bot supports three entry points so people can capture a task in whatever way fits the moment.

1.1 Capture via @-mention

> ***As a contributor**, I want to mention the bot in any channel and describe a task, **so that** the task is captured without me opening another tool.*

Flow:

1. User writes `@bot need to prepare presentation` in a channel.
2. The bot reads the message, extracts title + due date + assignee from the text.
3. The bot replies in the same thread with a **draft card** showing the extracted fields and three buttons: *Accept*, *Edit*, *Reject*.
4. User clicks *Accept*. The card morphs into a confirmed task card.
5. The task is now in the system; a row appears in the Google Sheet.

1.2 Capture from a voice note

> ***As a contributor on the go**, I want to record a Slack voice message describing a task, **so that** the bot transcribes it and creates the task without me typing.*

Flow:

1. User records a voice note in Slack.
2. Bot downloads the audio, transcribes it (OpenAI Whisper), and feeds the transcript into the same extraction pipeline as text messages.
3. Bot replies with the same draft card as in flow 1.1.
4. User Accepts / Edits / Rejects.

1.3 Passive capture (the bot notices for you)

> ***As a contributor in a busy channel**, I want the bot to notice when someone describes a task ("we need X by Friday"), **so that** I don't have to remember to capture it manually.*

Flow:

1. Someone writes in a channel where the bot is present: *"надо подготовить отчёт к понедельнику"*.
2. Bot reads the message, decides "this looks like a task", and posts a draft card with the extracted fields plus the three buttons.
3. Whoever cares (often the implied owner or the message author) clicks *Accept*, *Edit*, or *Reject*.
4. If nobody reacts in some time, the draft simply expires — nothing is created without a human click.

Feature 2 — Task lifecycle

Once a task is confirmed, every status transition is one click on the card. The lifecycle is deliberately simple: `backlog → todo → in_progress → done`. There are also two cross-cutting actions — *Cancel* (un-schedule) and *Delete* (remove).

2.1 Start working

> ***As the task owner**, I want to mark a task as started, **so that** my team sees what I'm actively working on.*

Flow:

1. Owner sees the task card (in the source channel or in their morning plan DM).
2. Owner clicks *Start*.
3. Card updates: status flips to `in_progress`, *Start* button replaced by *Mark done*, started timestamp recorded.
4. All subscribers of the task get a DM notification.
5. The Google Sheet row is updated.

2.2 Complete a task

> ***As the task owner**, I want to close a task and optionally attach the result, **so that** the team has evidence of what was done.*

Flow:

1. Owner clicks *Mark done* on the task card.
2. A small modal opens with two optional fields: *Artifact URL* and *Or description*.
3. Owner can either fill one of them and click *Complete*, or leave both empty and click *Complete* anyway.
4. Status flips to `done`. Card buttons collapse to read-only state.
5. Subscribers get a DM. Sheet row reflects `done` + the artifact.

2.3 Cancel — "I won't get to it now"

> ***As the task owner**, I want to drop a task back into the queue without closing it, **so that** I can come back to it later without the manager thinking I've abandoned it.*

Flow:

1. Owner clicks *Cancel* on the task card.
2. Bot decides where the task goes:
   - if its `due_date` is within this week (Mon–Sun) → back to `todo`;
   - otherwise → back to `backlog`.
3. Card refreshes with the new status.
4. Subscribers get a DM about the status change.
5. Sheet row is updated, with `reason="cancelled"` recorded in the task's status history.

2.4 Delete

> ***As the task owner or admin**, I want to remove a task entirely with confirmation, **so that** I can clean up duplicates or mistakes without losing the audit trail.*

Flow:

1. Owner / admin clicks *Delete* on the task card.
2. A confirmation modal appears: *"This will delete task #N — title. The task is hidden from the UI immediately; the row stays in the audit log."*
3. User clicks *Delete* in the modal (or *Cancel* to back out).
4. Card is replaced by a tombstone: *":wastebasket: Task #N — title deleted by @actor"*.
5. Task disappears from every digest / plan / list / Google Sheet (the sheet row's status flips to `deleted` and the timestamp is filled).

Feature 3 — Edit any field

> ***As the task owner or admin**, I want to edit any field of an existing task, **so that** I can correct a wrongly-extracted owner, change priority, or update the deadline.*

Flow:

1. Owner / admin clicks *Edit* on the task card.
2. A modal opens prefilled with all current fields: title, description, owner, priority, due date + time, start date + time, category, recurring schedule.
3. **Owner dropdown** lists every person the bot has seen in the workspace (real names from Slack profiles, never raw usernames).
4. User adjusts fields and clicks *Submit*.
5. Card refreshes with the new values; sheet row updates.

Feature 4 — Subscriptions

Subscribers track work without owning it.

4.1 Subscribe to someone else's task

> ***As a stakeholder**, I want to subscribe to a colleague's task, **so that** I'm notified when its status changes — without owning the work.*

Flow:

1. User opens any task card where they're not the owner.
2. User clicks *Subscribe*.
3. Bot DMs the user a confirmation with a copy of the task card.
4. From then on, every status change of that task posts a follow-up DM in the same thread (one task = one DM thread, so notifications don't fragment).
5. The button toggles to *Unsubscribe* on the card.

4.2 Manage all subscriptions in one place

> ***As a busy subscriber**, I want a single screen to see and prune what I'm subscribed to, **so that** I can clean up old interests in one go.*

Flow:

1. From the daily digest DM, user clicks *Manage subscriptions*.
2. A modal opens listing every task they subscribe to.
3. Each row has an *Unsubscribe* button.
4. Click → that task drops out of the list immediately.

Feature 5 — Daily plan

The daily plan is a two-step nudge: evening triage, morning execution.

5.1 Evening — review tomorrow's plan

> ***As a contributor**, I want a quick review of what's on my plate tomorrow, **so that** I can drop unrealistic items before bedtime.*

Flow:

1. At 18:00 local time, bot DMs the user a card titled *"Plan for <tomorrow's date>"*.
2. The card lists each candidate task with a *Skip* button on the right.
3. At the bottom: *Approve plan* button (optional) and a *Tracking* section listing tasks the user subscribes to.
4. User clicks *Skip* on each task they won't get to. Each click drops that one task from tomorrow's plan.
5. User can click *Approve plan* to confirm explicitly — but it's **optional**. The morning plan runs whether they did or not.

5.2 Morning — execute today's plan

> ***As a contributor**, I want my plan ready when I open Slack in the morning, **so that** I can start working immediately.*

Flow:

1. At 09:00 local time, bot DMs the user a *Today* card.
2. Each surviving task from yesterday's plan appears as a full task card with a *Start* button.
3. *Tracking* section shows tasks they subscribe to.
4. If the user **didn't** click *Approve plan* the night before, the DM gets a small note at the top: *":memo: Plan wasn't explicitly approved last night — running as-is."* — so the user knows what they're looking at.
5. User clicks *Start* on whatever they begin with.

Feature 6 — Reminders & digests

Five separate reminder channels. Each runs on its own schedule and is idempotent (one notification per (user, day), no spam on retry).

6.1 Morning digest (per subscriber)

> ***As a contributor**, I want a morning summary of what I own and watch, **so that** I know what to focus on today.*

Flow:

1. Each morning, bot DMs every subscriber a digest with three sections: *Today*, *Approaching deadlines (next 2 days)*, *Overdue*.

6.2 Weekly plan (Sunday)

> ***As a contributor**, I want a heads-up on Sunday evening about the week ahead, **so that** I plan my week before Monday morning.*

Flow:

1. Sunday 20:00, bot DMs every owner of `backlog` tasks due in the coming week with the list.

6.3 Thread reminders (10:00 weekdays)

> ***As a contributor**, I want a soft public nudge in the source thread of every active task, **so that** stakeholders see the task isn't forgotten and I'm reminded to update it.*

Flow:

1. Weekday 10:00, bot posts a one-line reminder in the source thread of each task in `in_progress` (and `todo` tasks due this week).
2. The text tags the assignee: *":raised_hand: <@owner> task #N — what's the progress?"*.
3. One ping per (task, day), so the thread doesn't get spammed if the cron retries.

6.4 Deadline reminders

> ***As an owner**, I want a DM 2 days before the deadline and on the day of overdue, **so that** I don't miss things.*

Flow:

1. Each morning, bot DMs the owner of every task whose due date is ≤ 2 days away or already overdue.
2. Includes the task title, due date, status.

6.5 Admin watch-list

> ***As an admin**, I want a daily snapshot of the team's active and stale work, **so that** I can prevent things from rotting.*

Flow:

1. Mornings, every admin gets a DM with three sections: *In progress* (across the whole team), *Overdue* (across the team).
2. Evenings, admins get *Tomorrow* (tasks due tomorrow by assignee) plus *Stale* (`in_progress` with no movement for 2+ days).

Feature 7 — Owner detection (no manual setup)

> ***As a contributor**, I want the bot to figure out who a task is for based on the message text, **so that** I don't have to fill the assignee field manually for every task.*

Flow:

1. When a task is being captured, the bot's LLM stage looks at the message + thread context.

2. If the message names someone explicitly ("Иван, сделай X" / "for @ivan" / "<@U123>"), bot picks that person — validated against the employees the bot knows about.
3. If nobody is named, bot quietly assigns the task to the message author (with an "(implicit)" badge on the card).
4. If the bot finds a name it doesn't recognise (e.g. "make Petya do it" but Petya isn't in the workspace), it asks in the source thread: *"I couldn't find Petya in the list. Who's the assignee?"*

Feature 8 — Employees directory (auto-discovery)

> ***As an operator**, I want the bot to learn the team automatically instead of me maintaining a list, **so that** new joiners are picked up without a redeploy.*

Flow:

1. On bot startup, it walks the entire Slack workspace (`users.list`) and indexes every member.
2. When the bot is added to a channel / DM it doesn't know yet, it walks `conversations.members` and indexes everyone in that room.
3. When someone new posts a message, the bot upserts their profile from `users.info` .
4. The Edit-modal owner dropdown reads from this directory in real time. Names shown use Slack's `real_name` first (then `display_name` , then user id) so cells / dropdowns never read "admin" instead of the actual person.

Feature 9 — Live Google Sheets sync

> ***As a manager**, I want every task and its current state in a shared Google Sheet I can filter / pivot / chart, **so that** I have one place to see all work without opening Slack.*

Flow:

1. Operator gives the bot a Google Service Account JSON and shares the target spreadsheet with the SA email.
2. Whenever a task is created, edited, started, completed, cancelled, or deleted, the bot **updates the same row** in the sheet within seconds.
3. Header row is written automatically on first sync — 21 columns in the bot's canonical order: id, title, description, owner, priority, category, start date/time, due date/time, recurring schedule, status, parent task id, source link, created/updated/ deleted timestamps, completion artifact.
4. Deleted tasks stay in the sheet with status `deleted` for audit.

Feature 10 — Audit & history

> ***As an admin or auditor**, I want a complete record of who did what and when, **so that** I can investigate any task's life from creation to closure.*

Flow:

1. Every status transition is recorded with `from` , `to` , `actor` , `reason` , `at` timestamps.
2. Every significant action (create, edit, delete, send digest, subscription change) is logged in `audit_logs` .
3. Every Slack message the bot sees is archived (text + transcript for voice).
4. Soft-deleted tasks are hidden from the UI but their history remains, including the `task_deleted` audit row with the actor and the status at the moment of deletion.

Feature 11 — Recurring tasks

> ***As an owner**, I want to mark a task as recurring on certain weekdays at certain times, **so that** ritual work (standups, reviews, etc.) doesn't need re-creating each week.*

Flow:

1. In the Edit modal, user selects weekdays (Mon, Wed, Fri…) in a multi-select.
2. Optionally sets a time range (e.g. 09:00–11:30).

3. Submits. The card now shows a `:repeat: Mon/Wed/Fri 09:00–11:30` line.
4. Selecting weekdays IS the toggle — there's no separate "Recurring" checkbox. Empty selection = not recurring.

Feature 12 — Telegram as a second source channel

The bot grows a second input channel: Telegram. Tasks captured from Telegram messages live in the **same** task table and the **same** Google Sheet as Slack tasks — managers see one combined view, filterable by source channel.

The Telegram side has a fundamentally different shape: the bot is NOT added to chats by itself. Instead, an upstream pipeline (out of scope for us) collects Telegram messages into a read-only Supabase view; we read from that view on a schedule and process every new message through the same intent pipeline as Slack.

12.1 — Continuous Telegram task capture

> ***As a contributor in a team Telegram chat**, I want my requests ("к понедельнику нужен отчёт") to land in the shared task tracker just like in Slack, **so that** my team has one home for tasks regardless of where the conversation happened.*

Flow:

1. Contributor writes a message in a Telegram chat that the upstream pipeline indexes into the Supabase view.
2. Every few minutes (cron-driven), the bot's Telegram ingest worker (`python -m ops.telegram_ingest`) reads new rows from the view, strictly after the last `(chat_id, message_id)` it has already processed.
3. Each new message goes through the same intent pipeline as a Slack message — language-agnostic detection, owner / date / title extraction, prefilter safety net.
4. If a task is detected, the bot creates a `Task` row with `source_kind = 'telegram'` . The row carries the original chat id, message id, reply-to id, and a `https://t.me/c/<chat>/<msg>` permalink (when the chat is a public super-group).
5. The new task appears in the same Google Sheet next to Slack tasks; admins see it in the morning watch-list digest. Source channel is filterable in the sheet via the `source_kind` column.

12.2 — Historical Telegram backfill

> ***As an operator deploying the bot in a team that's already used Telegram for months**, I want to import the entire Telegram history at once, **so that** the bot starts with the team's full backlog instead of only seeing future messages.*

Flow:

1. Operator runs `python -m ops.migrate_telegram_history` once, optionally with `--dry-run` first to preview counts.
2. The script walks the entire Supabase view from oldest to newest in batches.
3. Each batch is processed inside a transaction; errors per-message are caught and counted, not aborting the run.
4. At the end, a summary line in the log: total seen, tasks created, no-action, errors.
5. The script is **idempotent**: re-running it picks up only messages added since the last run (the same `(chat_id, message_id)` bookmarks are reused).

12.3 — Bot-as-listener mode

> ***As an operator who can't (yet) hook into the team's existing Telegram archive**, I want to add the bot to a chat / group and have it capture messages directly via the Bot API, **so that** I can test the whole pipeline end-to-end without waiting for the upstream Supabase setup.*

Flow:

1. Operator sets `TELEGRAM_BOT_TOKEN` in the env file.
2. **One-time BotFather step**: `/mybots → bot → Bot Settings → Group Privacy → Turn off` . Without this the bot only sees `/commands` and direct mentions in group chats.

3. Operator adds the bot to a chat or group as a normal member.
4. A separate container runs `python -m ops.telegram_listener` — long-poll on Telegram's Bot API, no public HTTP endpoint exposed, just outbound 443.
5. Every new message in those chats lands in the same `tasks` table with `source_kind = 'telegram'`, sharing `processed_ telegram_messages` bookmarks with the Supabase ingest path. Same Google Sheet, same admin digests.

The two ingest paths run in parallel and dedupe via the (chat_id, message_id) primary key — a message captured by either path is indistinguishable in the DB once written.

12.4 — Private task card with buttons (DM only — group stays clean)

> ***As a Telegram contributor in a group**, I want the task card to be visible only to the people who care about it — me (the author), the assignee, and admins — so that the rest of the group doesn't see noise from every captured task.*

Flow:

1. The listener captures a message and creates a task (see 12.3).
2. The bot **DMs** the card to a small recipient set:
   - the message author;
   - the assignee, if the LLM resolved a different owner;
   - every Telegram admin from `TELEGRAM_ADMIN_USER_IDS`.

   The source group / chat does **not** get a copy. Telegram doesn't do per-recipient visibility within a group, so the only privacy-preserving option is to skip the group post entirely.
3. Each recipient sees a keyboard rendered from THEIR perspective:
   - the author always sees Edit / Cancel / Subscribe;
   - the owner sees Start / Mark done / Edit / Cancel / Delete;
   - admins see Edit / Cancel / Delete on every task.
4. Tapping a button drives the task — the change applies once, and the bot edits **every delivered DM** so author / owner / admin all see the same state without re-fetching anything:
   - **Start** flips backlog/todo → in_progress.
   - **Mark done** opens the optional-artifact reply (see 12.5).
   - **Cancel** routes back to todo (this week) or backlog (later).
   - **Subscribe / Unsubscribe** toggles for bystanders.
   - **Delete** soft-deletes; every card flips to a tombstone line.
   - **Edit** opens the key=value reply conversation (see 12.5).
5. The same Google Sheet row updates after every change.

Important for the operator: Telegram's Bot API can DM only users who have already started a private chat with the bot (sent `/start` or any DM). Recipients who never started the bot won't get a card — the bot logs `telegram_card_dm_failed` and moves on. Ask team members to `/start` the bot once.

Permissions match the Slack card: only the owner (or admin) can do destructive things (Mark done / Cancel / Delete / Edit); bystanders can subscribe.

12.5 — Mark done + Edit via reply conversation

> ***As a Telegram task owner**, I want the same Edit and Mark-done- with-artifact flows as Slack — without modal dialogs Telegram doesn't support, **so that** I can drive a task end-to-end without switching to Slack.*

Mark done flow:

1. Owner taps *Mark done* on the card.

2. Bot replies in the chat: *"Optional: reply with a link or short note. Or `/skip` to complete without."* The reply triggers a force-reply UX in the user's client.
3. User replies with one of:
   - a URL (e.g. `https://drive.example.com/file`) → saved as `kind=url`;
   - free text → saved as `kind=text`;
   - `/skip` → no artifact.
4. Bot transitions the task to *done*, refreshes the original card in place, and updates the Google Sheet row.

Edit flow:

1. Owner / admin taps *Edit*.
2. Bot replies with the current values formatted as `key=value` pairs and a list of allowed keys: `title`, `description`, `priority`, `due`, `due_time`, `start`, `start_time`, `category`, `owner`.
3. User replies with one or more `key=value` lines for the fields they want to change. Empty value clears the field. Invalid values (e.g. `priority=critical`) are silently ignored.
4. Bot applies the changes, refreshes the card, drops the `owner_assumed` flag, updates the sheet.

The matching between bot prompts and user replies uses Telegram's `reply_to_message_id` field and a 10-minute in-memory TTL — unrelated chat traffic never accidentally triggers a handler.

12.6 — DM digests, daily plan, reminders for Telegram users

> ***As a Telegram user who owns tasks**, I want the same morning digest / daily plan / weekly plan / deadline / thread reminders that Slack users get, **so that** my channel of choice gets the same coverage.*

What's covered:

- *Morning digest* — DM with Today / Approaching (2 days) / Overdue per Telegram owner.
- *Daily plan* — evening heads-up DM, morning execution DM with today's tasks. Approve is optional (matches FR-CR-04-25).
- *Weekly plan* — Sunday evening DM listing next week's backlog.
- *Deadline reminders* — DM the owner of any task ≤ 2 days from due (or already overdue).
- *Thread reminders* — daily nudge posted in the source Telegram chat under the original message.
- *Admin watch-list* — DM each TG admin with the team-wide *In progress* + *Overdue* lists.

Routing rule: numeric user ids → Telegram, others (Slack shape `U…` / `W…`) → Slack. A Slack subscriber never gets a Telegram DM and vice-versa. Per-user / per-day idempotency lives in `audit_logs` under category prefix `telegram_*`.

TG admins are configured via the `TELEGRAM_ADMIN_USER_IDS` env var (comma-separated numeric ids). Same role as Slack admins: edit / cancel / delete any task, plus admin watch-list digest.

Operator setup — the cron schedule mirrors the Slack one, just doubled with a Telegram call per slot:

```
09:00  python -m ops.telegram_digest --type morning-digest
09:00  python -m ops.telegram_digest --type plan-morning
10:00  python -m ops.telegram_digest --type thread-reminders   (Mon-Fri)
18:00  python -m ops.telegram_digest --type plan-evening
20:00  python -m ops.telegram_digest --type weekly             (Sundays)
every 6h python -m ops.telegram_digest --type deadlines
09:00  python -m ops.telegram_digest --type admin-watchlist
```

---

5. Under the hood — how it's built

Hosting & deployment

- **One Google Cloud VM** (Compute Engine, region `europe-west1`) hosts the entire stack. No Kubernetes, no autoscaling — single box, Docker, restart-on-failure.

- Three Docker containers run side by side, on a private Docker network so they talk to each other but not to the public internet:
  - `slack-task-bot` — the application itself. Python 3.11. Restart policy `unless-stopped`, so a crash brings it back.
  - `slack-task-db` — PostgreSQL 16 (Alpine). Persistent volume so data survives container restarts.
  - `slack-task-db-proxy` — small `socat` container exposing the DB on a non-default host port for read-only inspection from outside (admin convenience, not used by the bot itself).
- Source code lives in a Git repository; deploys are `git pull` → `docker build` → `docker run`. No CI/CD pipeline yet.

How the bot talks to Slack

- **Slack Bolt for Python** is the framework that handles incoming events, button clicks, modal submits, and slash commands.
- The bot connects to Slack over **Socket Mode** — an outbound WebSocket from the bot to Slack — so we don't have to expose a public HTTP endpoint, no inbound firewall rules, no SSL cert. The VM can sit fully behind a firewall; only outbound 443 needs to work.
- A Slack App **manifest** is checked into the repo (`ops/ slack-manifest.yaml`) — it pins the bot's scopes, shortcuts and events as code, so re-installing the app from scratch is a copy- paste away.

Data layer

- **PostgreSQL 16** is the single source of truth. Tables cover:
  - tasks (with status history, subscriptions, sync state);
  - employees directory;
  - daily plan items;
  - draft pipeline (intent inference, action drafts, context snapshots);
  - audit log (every significant action keyed for indexed lookup);
  - full Slack message archive (with voice transcripts).
- **Alembic** versions every schema change — currently at migration `0013`. Rolling back to any historical state is one command.
- SQLite is used in tests (in-memory) so the test suite runs without a Postgres instance — same SQLAlchemy code path.

Intelligence layer (LLM + speech)

- **LangGraph** drives a small state machine that runs every captured message through four focused stages: *detect → describe → owner → date → assemble*. Each stage is a separate LLM call with a tiny tool schema, which is much more reliable than one mega-prompt.
- **OpenAI** (default `gpt-4o-mini` for routing/title/owner, `gpt-4o` for date) is the primary LLM. **Anthropic Claude** is available as a fallback / alternate provider via a single env setting (`LLM_PROVIDER=anthropic`). Both go through the same `LLMBackend` abstraction.
- **OpenAI Whisper** transcribes Slack voice notes. The transcript is fed into the same pipeline as text messages.
- A small **rule-based prefilter** acts as a safety net: if the LLM returns `no_action` on an obviously task-shaped message, the prefilter forces a draft.
- **No data leaves the bot's process** except the literal LLM / Whisper API calls — no analytics, no telemetry to third parties.

External integrations

- **Google Sheets API** — live sync. Auth via a **Service Account** (JSON key mounted into the container as `/app/secrets/sa.json`). The spreadsheet is shared with the SA's email as Editor; the SA has no other access.
- **Google Tasks API** — optional, off by default. Same auth.

Reliability primitives

- **Idempotency.** Every digest, plan, and reminder is keyed `(category, action, actor, date)` in the audit log. Re-running the cron on the same day is a no-op.

- **Retries.** Outbound HTTP (Slack, Sheets) is wrapped in `tenacity` with exponential backoff for transient failures. Slack rate-limits (HTTP 429) are honoured automatically.
- **Soft delete.** Deleted tasks set `deleted_at` and are filtered out of every query, but the row stays for audit. Nothing is ever hard-deleted from `tasks`.
- **Best-effort sync.** Google Sheets sync is wrapped so a Google outage logs a warning but never breaks the Slack interaction — task state in Postgres is always authoritative; Sheets is a derived view.
- **Per-stage isolation.** A failure in one LangGraph node (e.g. the owner stage timing out) leaves that field empty rather than aborting the whole capture; the follow-up loop fills it in later.

Secrets & access

- All secrets live in `/root/slack-task/.env` (loaded at container start via `--env-file`):
  - `SLACK_BOT_TOKEN`, `SLACK_APP_TOKEN`, `SLACK_SIGNING_SECRET`
  - `OPENAI_API_KEY`, optionally `ANTHROPIC_API_KEY`
  - `DATABASE_URL`
  - `SECRETS_ENCRYPTION_KEY` (Fernet key for any OAuth tokens stored in DB; not used in the SA path)
  - `GOOGLE_SHEETS_SPREADSHEET_ID`, `GOOGLE_SHEETS_TAB_NAME`, `GOOGLE_SERVICE_ACCOUNT_JSON_PATH`
  - admin Slack user ids, allowed-owner overrides
- The Service Account JSON is mounted into the container as a read-only volume — never baked into the image.

What runs on cron

The bot's daily / weekly notifications are kicked off by `cron` on the host (timezone `Europe/London`):

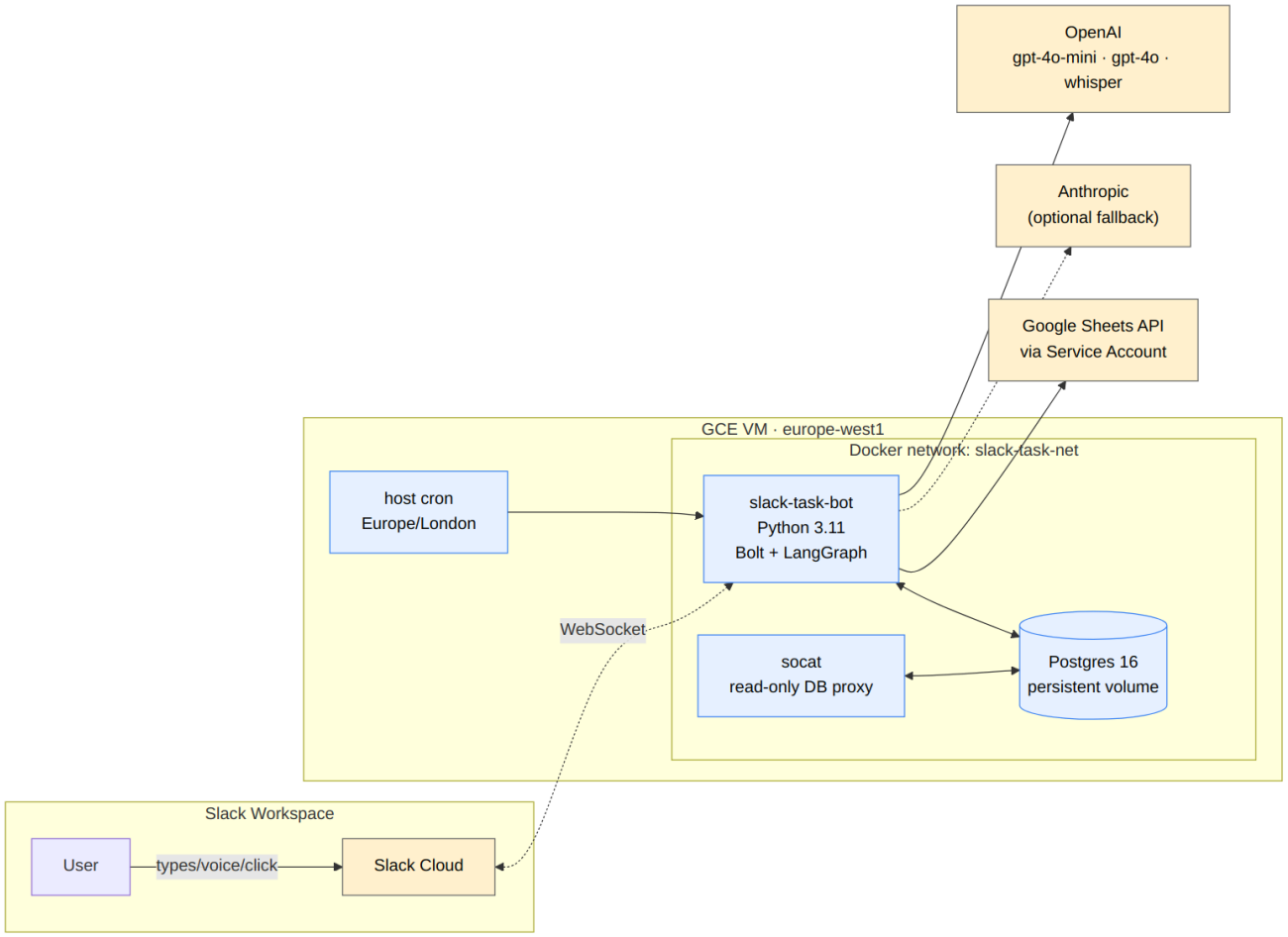
Time	Job	What it does
09:00	morning digest + morning plan	DM each user their plan + dailies
10:00 (Mon-Fri)	thread reminders	nudge each open task in its source thread
18:00	evening plan	DM each user the next-day plan
20:00 (Sun)	weekly plan	DM each owner their next week
every 6 h	deadline reminders	DM owners whose deadlines are ≤ 2 days

Each job is its own `python -m ops.send_digest --type ...` call inside the bot container; idempotency means retries are safe.

Diagrams

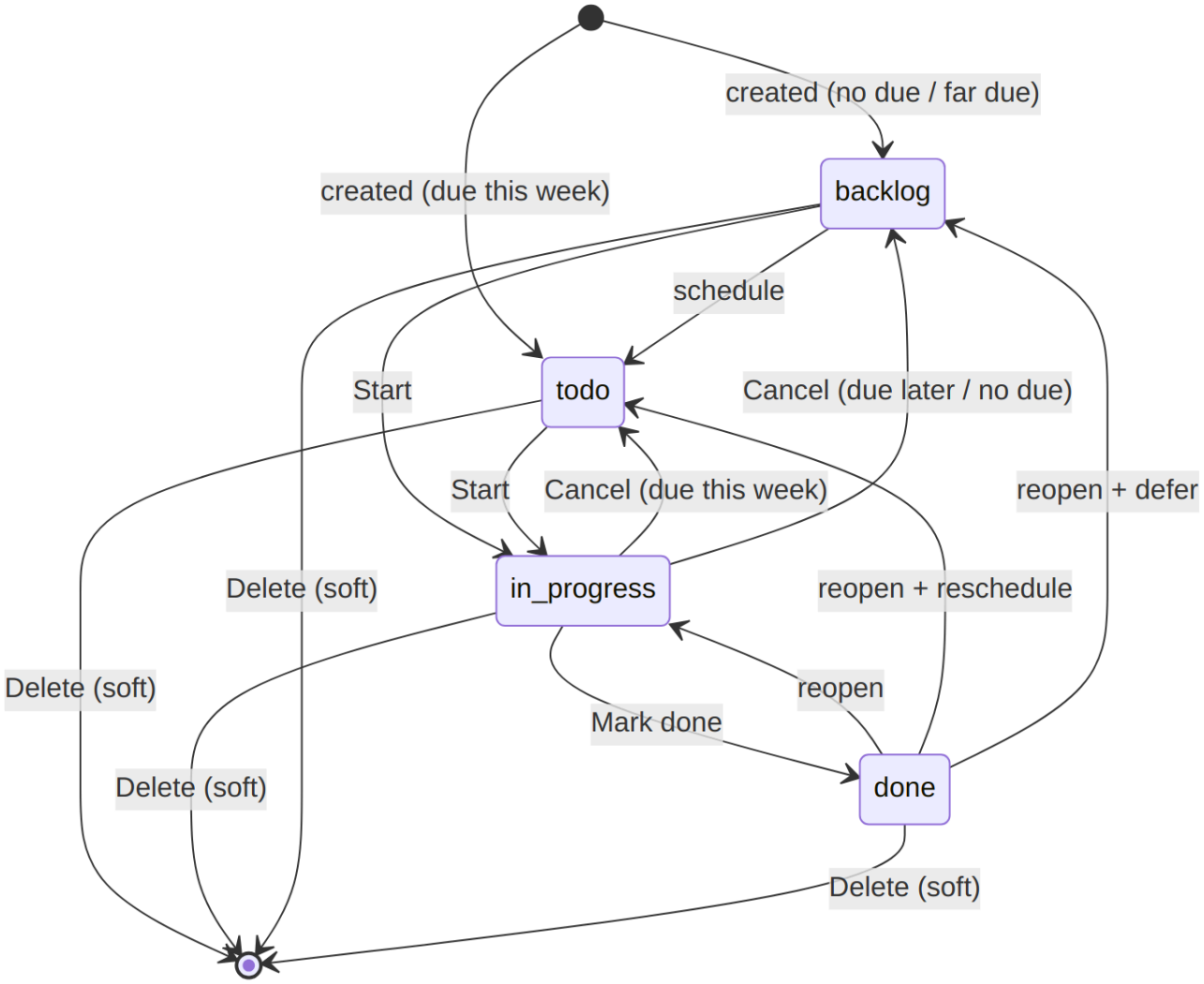
System architecture

How the bot, its database, and external services fit together on a single GCE VM. Slack speaks to the bot over an outbound WebSocket (Socket Mode), so no public HTTP endpoint is needed.



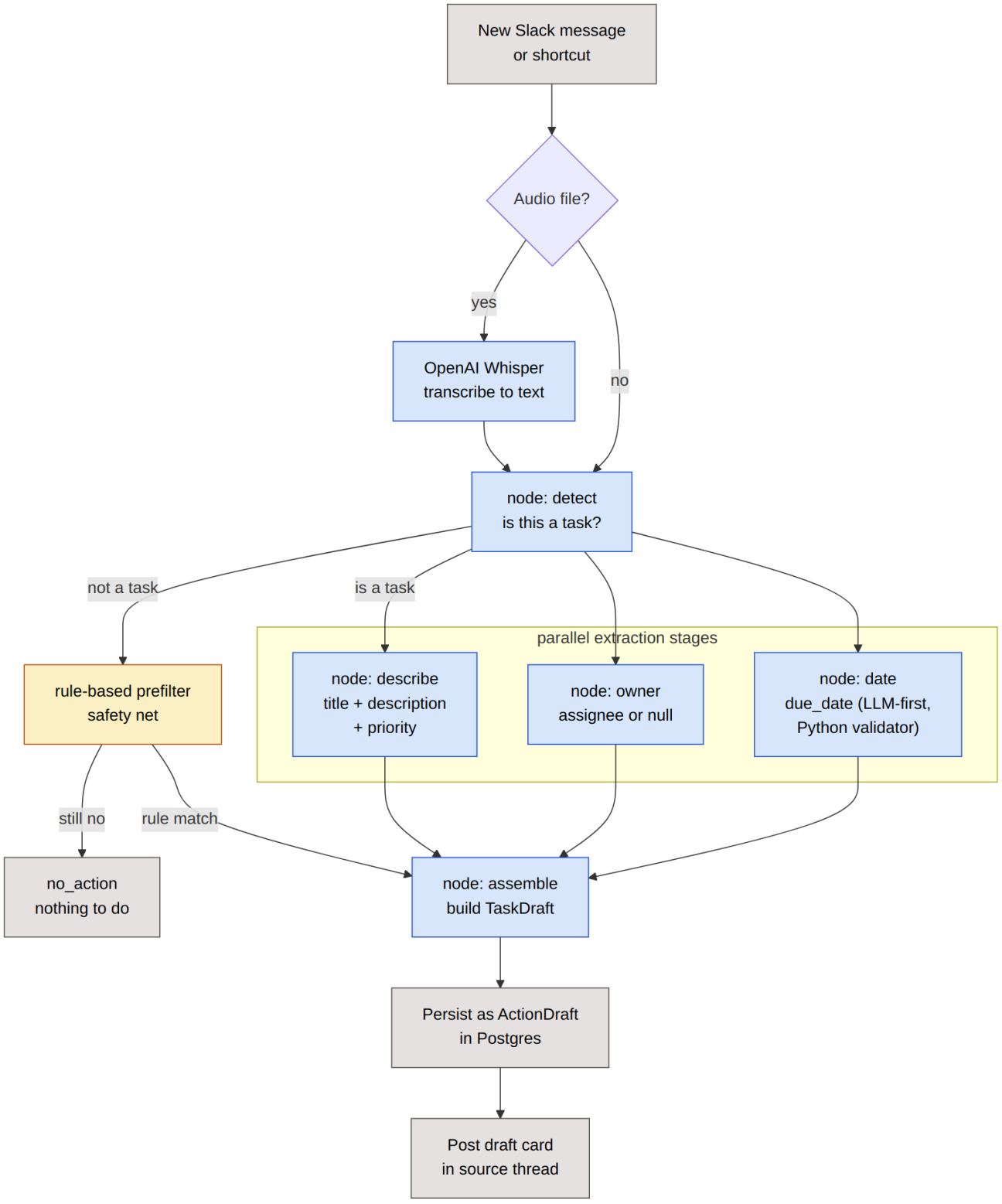
Task lifecycle

The four-state task model with all legal transitions, including the *Cancel* back-edges that route by `due_date` and the *Delete* soft- removal terminator.



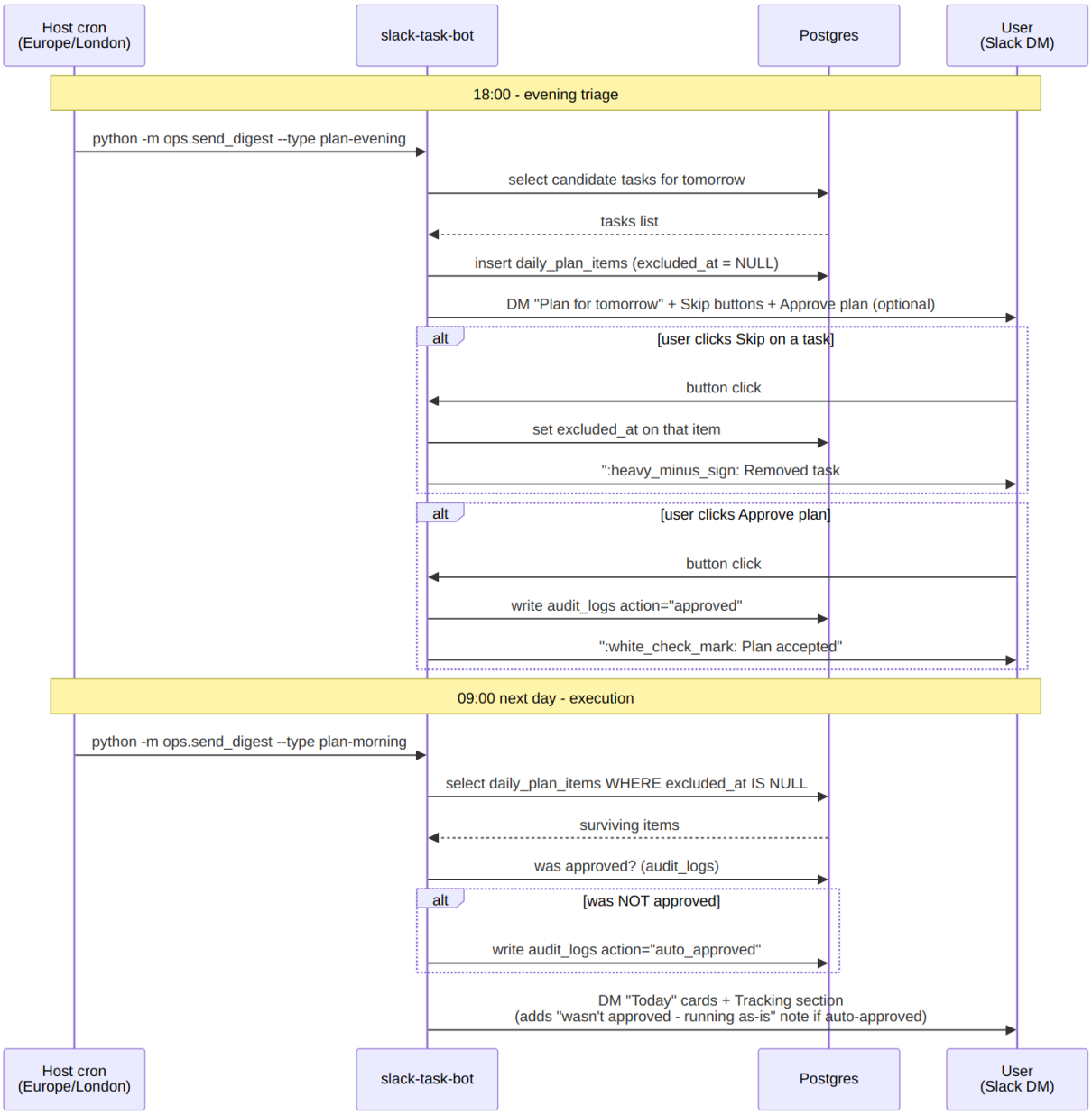
Intent extraction pipeline

How a single Slack message (or voice note) becomes a draft task. Detection gates the parallel extraction stages; a rule-based prefilter is a safety net for obviously task-shaped messages the LLM might miss.



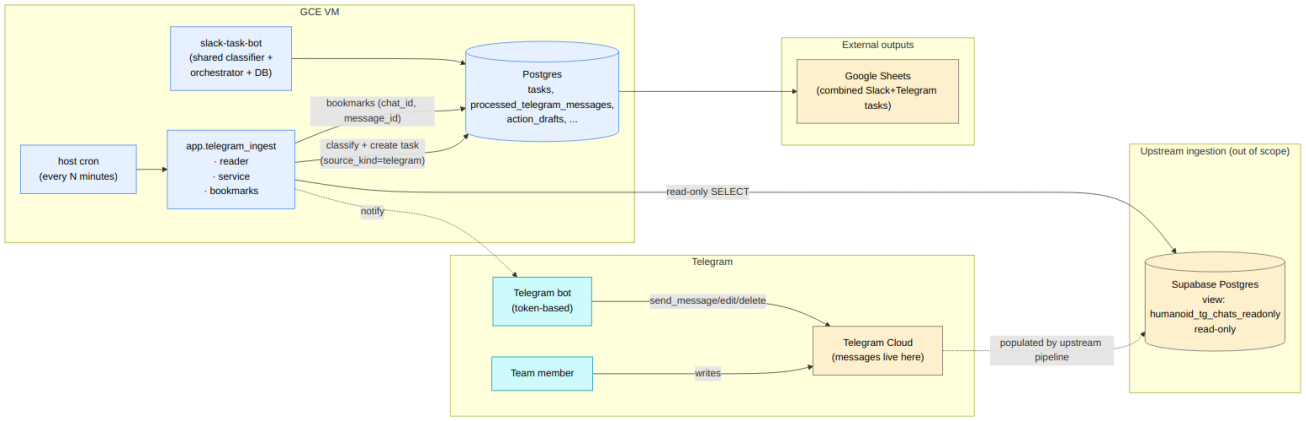
Daily plan flow

End-to-end sequence of the evening triage at 18:00 and the morning execution at 09:00 the next day, including the optional *Approve* shortcut and the *auto-approved* path when the user didn't explicitly confirm.



Telegram channel

How Telegram messages enter the system from a separate read-only Supabase view, get processed by the same intent pipeline as Slack, and land in the same task DB + Google Sheet. The Telegram bot itself only handles outbound notifications back into Telegram — it isn't a peer in the chat the way the Slack app is.


GCE VM
slack-task-bot
(shared classifier +
orchestrator + DB)
Postgres
tasks,
processed_telegram_messages,
action_drafts, ...
bookmarks (chat_id,
message_id)
host cron
(every N minutes)
app.telegram_ingest
· reader
· service
· bookmarks
classify + create task
(source_kind=telegram)
notify
External outputs
Google Sheets
(combined Slack+Telegram
tasks)
Upstream ingestion (out of scope)
read-only SELECT
Supabase Postgres
view:
humanoid_tg_chats_readonly
read-only
Telegram
Telegram bot
(token-based)
send_message/edit/delete
Telegram Cloud
(messages live here)
Team member
writes
populated by upstream
pipeline